Apartment Rental Management System (ARMS) – Technical & Product Documentation

Table of Contents

1. 1. Executive Summary
2. 2. Product Description and Vision
3. 3. Core Features and Capabilities
4. 4. Technical Architecture and Infrastructure
5. 5. Technology Stack
6. 6. Security Architecture
7. 7. Role and Authorization Management Model
8. 8. Clearance / Access-Level Model
9. 9. User Experience and Interface
10. 10. Installation and Deployment
11. 11. Integration and Scalability
12. 12. Competitive Analysis and Differentiation
13. 13. Licensing and Commercial Model
14. 14. Technical Requirements
15. 15. Contact and Support

1. Executive Summary

The Apartment Rental Management System (ARMS) is an advanced digital platform for property rental operations across buildings and portfolios. It centralizes apartments, tenants, contracts, payments, maintenance, and notifications in one place. Built with modern web technologies, ARMS offers a user-friendly, highly customizable control panel and seamless integrations with payment and messaging services.

2. Product Description and Vision

2.1 Problem Domain

- Fragmented tools (spreadsheets, chats) cause lost documents and inconsistent records.
- Delayed payments due to missing reminders and manual follow-up.
- Weak maintenance tracking across units and technicians.
- No complete communication history between landlord and tenant.
- Difficulty identifying overdue balances and portfolio-wide KPIs.

2.2 Solution: ARMS

- End-to-end rental lifecycle: unit onboarding → tenant onboarding → contract → renew/terminate.
- Payments dashboard with paid / overdue / upcoming status and automated reminders.
- Maintenance module with work orders, assignments, SLAs, and status tracking.
- Notifications engine for contracts, payments, and maintenance events.
- Full audit trail of every action by users and services.

2.3 Target Audience

- Real-estate companies and property managers.
- Residential complexes and communities.
- Real-estate offices and brokers.
- Individual landlords and small portfolios.
- Companies renting offices and commercial spaces.

3. Core Features and Capabilities

- Property & Unit Management: buildings, apartments, occupancy status, unit metadata.
- Tenant Management: profiles, KYC, contracts, history, documents.
- Contract Lifecycle: creation, e-sign, renewals, terminations, notices.
- Payment Tracking: invoices, receipts, partial payments, refunds.
- Maintenance Requests: ticketing, technician assignment, parts, SLA tracking.
- Notifications & Alerts: payment reminders, contract expiry, delays, maintenance.
- Audit Logs: immutable event log for compliance and troubleshooting.
- Reports & Analytics: occupancy, revenue, arrears aging, maintenance performance.

4. Technical Architecture and Infrastructure

4.1 Architecture

- Frontend: React (SPA)
- Backend: Node.js or Supabase Edge Functions
- Database: PostgreSQL
- Infrastructure: Docker-ready containers

4.2 Layered Architecture

- Presentation Layer: React + Tailwind CSS
- Application Layer: state management, validation, workflows
- Backend Layer: REST APIs, authentication, billing
- Database Layer: PostgreSQL with row-level access
- Infrastructure Layer: Dockerized services and CI/CD

4.3 Data Flow

- User signs in.
- System verifies role and permissions.
- Requested data is retrieved from APIs/DB.
- Data is displayed in the UI.
- Action is appended to the audit log.

5. Technology Stack

5.1 Frontend Technologies

- React
- TypeScript
- Vite
- Tailwind CSS

5.2 Backend and Database

- Node.js
- Supabase or Express API
- PostgreSQL

5.3 Code Distribution & Tooling

- GitHub Actions (CI/CD)
- Docker
- ESLint / Prettier

6. Security Architecture

6.1 Authentication

- Secure token-based login with short-lived JWTs and refresh tokens.
- Optional Two-Factor Authentication (2FA).
- Session revocation and device/session management.

6.2 Authorization

- Role-Based Access Control (RBAC).
- Granular permissions per module (units, tenants, contracts, billing).
- Portfolio/building-scoped access and least-privilege enforcement.

6.3 Data Security

- Field-level encryption for sensitive fields (e.g., payment tokens).
- PostgreSQL Row Level Security (RLS) for record isolation.
- Environment variables outside source; secrets stored in a vault.

6.4 Auditing and Traceability

- Immutable audit logs for all critical actions.
- Timestamped history for tenants, contracts, payments, and maintenance.
- Admin dashboards to review and export events.

6.5 Backup and Disaster Recovery

- Automated daily backups with retention.
- Point-in-time recovery support.
- Geo-replicated storage (if cloud-enabled).

6.6 Secure Development Practices

- Static typing via TypeScript.
- CI/CD security checks and dependency scanning.
- Formal vulnerability disclosure and patching process.

7. Role and Authorization Management Model

7.1 Admin

- Full system permissions.
- Manage buildings, units, tenants, and contracts.
- Access to all financial and operational reports.

7.2 Property Manager

- Handle tenant operations and communications.
- Manage maintenance requests and schedules.
- Process contract updates and notices.

7.3 Accountant

- Access to billing, rent payments, invoices, and ledgers.
- Reconcile payments and generate statements.

7.4 Tenant

- Restricted access to personal contracts and payments.
- Online rent payment (if enabled).
- Submit maintenance requests.

8. Clearance / Access-Level Model

ARMS uses a multi-layer access model inspired by enterprise security practices:

L0 – Public

Marketing/landing content and general information.

L1 – Internal

Company internal data not shared with tenants.

L2 – Confidential

Contracts, tenant identity details, and financial documents.

L3 – Restricted

Payment-related data, invoices, and banking operations.

L4 – Highly Restricted

Complete financial reporting, administrative logs, and sensitive internal data.

9. User Experience and Interface

9.1 Modern Responsive Design

- Tailwind-based, mobile-optimized.
- Adaptive breakpoints for desktop, tablet, and mobile.

9.2 Theme System

- Light & Dark modes with instant switching.
- Configurable color palette and branding.

9.3 Accessibility

- WCAG 2.1 compliant components.
- Keyboard navigation and screen reader support.

9.4 Notification System

- Color-coded toast notifications: success, warning, error.
- Real-time updates for payments, contracts, and maintenance.

10. Installation and Deployment

10.1 Docker Deployment

- Production-ready Dockerfile.
- Multi-container environments via Docker Compose.

Command:

```
docker-compose up --build
```

10.2 Local Development

- Node.js 20+

- npm 10+
- Run: npm install → npm run dev → npm run build

10.3 Cloud Deployment

- Frontend: Vercel / Netlify
- Backend: Supabase / Render / Railway

11. Integration and Scalability

- Email/SMS providers for notifications and reminders.
- Online payment gateways for rent collection.
- Webhooks for CRM/ERP integrations.
- Horizontally scalable stateless services and managed PostgreSQL.
- Multi-team and multi-property support.

12. Competitive Analysis and Differentiation

Feature	ARMS	Traditional Rental Tools	Generic Property Apps
Online Payments	✅	❌	Partial
Contract Lifecycle (Create→Renew→Terminate)	✅	❌	Partial
Maintenance Tracking	✅	❌	Partial
Multi-Building Support	✅	❌	Partial
Detailed Reporting	✅	❌	Partial
Cloud-Ready & Dockerized	✅	❌	Partial

13. Licensing and Commercial Model

Starter

- Up to 50 units
- 1 building
- L0–L2 access
- 30-day audit log
- Email support, 99% SLA

Professional

- Up to 250 units
- Up to 5 buildings
- L0–L3 access
- 1-year audit log
- On-premise support option, 99.5% SLA

Enterprise

- Unlimited units/buildings
- L0–L4 access
- Unlimited audit retention
- Dedicated 24/7 support
- 99.9% SLA

14. Technical Requirements

Server Side

- Node.js 20+
- npm 10+
- Docker (recommended)
- Managed PostgreSQL 15+

Client Side

- Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- JavaScript enabled
- Minimum 1280×800 resolution

Environment Variables

- VITE_APP_NAME – Application name (default: ARMS)
- VITE_DEFAULT_THEME – Default theme (light/dark)
- PAYMENTS_PROVIDER_KEY – Payment gateway API key (secret)
- SMS_API_KEY – SMS provider key (secret)

15. Contact and Support

Support: support@arms.app

Website: www.arms.app